# Messaging & Communications Decision Guide

Use this guide to design communication among patients, care teams, and external participants. Work through thread structure, routing, read state, attachments, automation, and delivery channels to choose the FHIR model and responsibilities for each messaging workflow.

*Companion to the Messaging & Communications docs.*

Start with the conversation's purpose and participants:

1. **Define conversations and content.** Choose thread participants and patient context, then add attachments, editing, or drafts where needed.
2. **Assign response ownership** when messages create work for a team. Define assignment, reassignment, escalation, and any automated follow-up.
3. **Choose delivery and read state.** Decide whether polling or live updates fit the workflow, what read state is needed, and which external channels support delivery or replies.
4. **Decide when messaging is a clinical encounter** if asynchronous care or billing is in scope.

Use the questions below to establish your use case, then follow the relevant sections. Add requirements beyond these examples where needed.

## Before you start

### Who is messaging whom?

- Patient ↔ provider
- Provider ↔ provider

- Care team internally (no patient in the thread)
- System/automated → patient (one-way notifications)
- Some combination of the above

*Why it matters: different participant types affect access control, thread structure, and whether a patient subject is always required on a thread.*

### What is the primary purpose of messaging in their product?

- Clinical care coordination (e.g. async visits, follow-ups)
- Patient engagement / support
- Internal operational communication
- Outbound notifications only (reminders, results)

*Why it matters: determines how much of the feature set is actually needed and whether async encounters/billing will come up.*

### Is this a new build or replacing something they have today?

- If replacing: what are they replacing, and what are the gaps that drove this?
- If new: is messaging central to the product or additive?

*Why it matters: existing systems often carry assumptions about data model or UX that will surface as requirements. Knowing this early prevents surprises.*

## Conversations and content

### How will threads represent participants and patient context?

**Questions:**

- Are threads always tied to a specific patient, or do you need internal/admin threads with no patient context?
- Are threads 1:1 only, or do you need group conversations with more than two participants?
- Do threads need to be tagged or categorized (e.g. by specialty, product line, urgency)? If so, do tags need to be combinable (e.g. a thread tagged both “cardiology” and “urgent”)?

**What the answers drive:**

| Situation | Approach |
|---|---|
| Threads always tied to a patient | Set `subject` on the thread header and each child message; each resource needs its own patient context for patient-compartment access |
| Internal/admin threads needed | `subject` is optional — don't assume it's always present in queries or UI |
| 1:1 only | Include both participants, including the creator, in the header's `recipient`; simpler read receipts (see Read state) |
| Group threads needed | Recipient list on thread header must include all participants including the thread creator |
| Tags/categories needed, single dimension | Use a single `category` entry on both the thread header and child messages |
| Tags/categories needed, multiple combinable dimensions | Use multiple `category` entries (e.g. one for specialty, one for credential level) — more maintainable but queries may need to match several categories |

Follow the thread data model: the header has no `payload` or `partOf`; child messages carry the content and link to the header through `partOf`. The header's `recipient` is the full participant list; a child's recipients may exclude its sender. A child does not inherit patient access through `partOf`. Enforce visibility with AccessPolicy on the resources themselves.

## How will attachments be stored and used?

**Questions:**

- What file types need to be supported?
- Do attachments need to be searchable or referenceable as clinical documents — e.g. queryable by patient or document type — or are they just files to download within a message?

**What the answers drive:**

| Situation | Approach |
|---|---|
| Files are just downloads within a message | Upload with `createAttachment` or `createBinary`, then put a `Binary/{id}` URL in the payload's `contentAttachment`. Set an appropriate Binary `securityContext`; do not embed file bytes in `Attachment.data` |
| Attachments need to be clinical documents (searchable, referenceable by patient/type) | Create a `DocumentReference` and attach via `contentReference` on the payload — file becomes a first-class searchable clinical document. Note: this is the default behavior of Medplum's React `ThreadChat` component. |
| Attachments are references to existing clinical resources (e.g. a lab result) | Use `contentReference` pointing at the relevant FHIR resource (e.g. `DiagnosticReport`) |

Give patient documents their own patient context and govern Binary access through securityContext. Changing the header's participants does not automatically change permissions on earlier messages, DocumentReferences, or files. Include those resources in membership-change and revocation checks.

## Can messages be edited, and where do drafts persist?

**Questions:**

- Can sent messages be corrected or retracted after sending?

- If so, does the original message content need to be discoverable via search (e.g. “show all edited messages”), or is it sufficient for the history to exist but not be directly queryable?
- Do users need to save a draft and return to it across sessions or devices, or is per-device/browser draft storage acceptable?

**What the answers drive:**

| Situation | Approach |
|---|---|
| Sent messages cannot be edited | No special handling needed |
| Sent messages can be corrected; edits need to be searchable | Retract-and-correct: mark original `status = entered-in-error`, create new message with corrected content linked via `inResponseTo` and tagged with a `correction` category. Corrections are discoverable via status and category filters. |
| Sent messages can be corrected; searchability of edits not required | In-place payload update via `patchResource`. Full version history still exists via `_history` but is not searchable. |
| Drafts only needed per browser/device | Store in browser `localStorage` keyed by thread ID — no server resources needed, but lost if user clears storage or switches devices |
| Drafts need to persist across devices/sessions | Store as `Communication` with `status = preparation`; update as user types (debounced); promote to `in-progress` with a `sent` timestamp when sent. Filter drafts out of thread message queries with `status:not=preparation`. |

## Response ownership

### How will responses be assigned, reassigned, and escalated?

**Questions:**

- Do messages get assigned to specific providers, to a pool/queue by role, or both?
- Can threads be reassigned or escalated after initial assignment?
- After reassignment, does the previous owner need to retain visibility of the thread?
- Do you need a structured audit trail of who owned a thread and why it was rerouted?

**What the answers drive:**

| Situation | Approach |
|---|---|
| Individual assignment only | Set `Task.owner` to a specific provider; retain `performerType` if a role requirement remains relevant |
| Pool/queue routing | Set `Task.performerType` by role type; clear `Task.owner`; providers claim from pool |
| Both | Use `performerType` for pool eligibility; set `owner` when claimed, retaining applicable role requirements |
| Reassign the same response obligation | Update the existing Task's owner with an `If-Match` version check. Add a new conversation participant through the membership workflow while preserving existing participants |

| Situation | Approach |
| --- | --- |
| Previous owner's visibility changes | Decide access through AccessPolicy and the membership workflow; review existing child messages and files as well as the header. Do not duplicate the Task just to preserve visibility |
| New, independently completed obligation | Create another Task with its own owner and completion criteria; both Tasks may focus on the same thread |
| Audit trail needed (free text) | Append to `Task.note` on each reroute with author and timestamp |
| Audit trail needed (structured/queryable) | Create a `Provenance` resource on each reroute with a coded reason — queryable via `Provenance?target=Task/{id}` |

Task ownership, conversation membership, and authorization are separate decisions. The application checks assignment eligibility and performs the updates; changing a Task does not automatically synchronize the thread. Use a transaction with version checks when the Task and header must change together. See Message Response Tracking and Routing.

## Which events or deadlines should trigger automation?

**Questions:**

- Are there events that should trigger automated messages (e.g. new lab result, appointment reminder)?
- Are there SLA or response-time requirements (e.g. all messages responded to within 4 hours)?
- Do providers have out-of-office or availability states that should trigger automatic replies or affect routing?
- Do you need to report on SLA compliance or response time metrics?

**What the answers drive:**

| Situation | Approach |
| --- | --- |
| Event-triggered automation (e.g. send message when lab result arrives) | Bot triggered by a Subscription on the relevant resource type |
| Recurring automation (e.g. flag threads with no response after N days) | Cron-scheduled Bot that scans for stale threads and creates reminder Tasks |
| Out-of-office / availability-based rerouting | Define messaging coverage or out-of-office rules; a Bot updates the existing Task when rerouting is required. If booking availability is part of the rule, calculate it from Schedules and busy/blocked Slots rather than looking for pre-created open Slots |
| SLA reporting needed | Task data (`authoredOn`, status change timestamps, `owner`, `priority`) should be exported to an analytics platform — FHIR search is not designed for aggregate SLA metrics |

Plan persisted subscriptions separately from live WebSocket listeners. Verify tenant scope and define idempotency, failure detection, and recovery using Planning Automations.

## Delivery and read state

### Are live updates required, or is polling sufficient?

**Questions:**

- Do new messages need to appear in real time without a page refresh?
- Is this needed for the patient-facing app, the provider-facing app, or both?

**What the answers drive:**

| Situation | Approach |
|---|---|
| Real-time updates needed | WebSocket subscriptions on the thread — requires this to be scoped and enabled for the project |
| Polling is acceptable | Simpler implementation; periodic re-fetch of thread messages |
| Notification only (e.g. badge, push) without live message rendering | Can use subscription to trigger a notification without requiring in-place message updates |

## What read state must be visible and queryable?

**Questions:**

- Are threads ever group conversations (more than 2 participants)?
- Do you need to query unread counts via API — e.g. a badge count or a list of all unread threads for a user?
- Do unread counts need to be tracked per-recipient in a group thread?

**What the answers drive:**

| Situation | Approach |
|---|---|
| 1:1 threads only | Option A: mark message `status = completed` when read; query unread via status filter. Simplest model. |
| Group threads, no API unread query needed | Option B: store per-participant read state as an extension on the thread header. Shows unread dot in UI; not searchable via API. |

| Situation | Approach |
| --- | --- |
| Group threads, API unread query needed | Option C: create a read-receipt Task per recipient per message; complete it when read. Supports badge counts and unread queries. Most complex. |

## Which external channels need outbound delivery or replies?

**Questions:**

- Which channels — SMS, email, other?
- Is this outbound-only (notify the patient) or does the patient reply back into the thread?
- If inbound replies are needed, how is the patient identified from an incoming message (e.g. phone number, email address)?
- Does the external provider (e.g. Twilio) have its own conversation ID that can be stored for thread matching?

**What the answers drive:**

| Situation | Approach |
| --- | --- |
| Outbound notifications only | App creates the Communication, then calls `executeBot` directly to send via the external channel — not Subscription-triggered, so errors surface synchronously |
| Inbound replies needed | Inbound webhook Bot receives the payload, identifies the patient, matches or creates a thread, creates a child Communication |

| Situation | Approach |
| --- | --- |
| External conversation ID available (e.g. Twilio conversation SID) | Upsert thread header by that identifier on first message; use it as conditional `partOf` reference on all inbound messages — most reliable thread matching strategy |
| No external conversation ID | Match thread by patient phone/email and open thread status; ambiguous cases need a defined fallback strategy |
| Round-trip (provider replies in-app, patient receives via SMS/email) | Store channel on thread header as `medium`; app creates the Communication then calls `executeBot` to dispatch outbound — Bot reads `medium` to route to the right channel |
| Webhook retry / duplicate messages likely | Use `createResourceIfNoneExist` with the provider's message ID as an identifier to prevent duplicate inbound Communications |

## When is messaging a clinical encounter?

**Questions:**

- Does the customer intend to bill for messaging-based care?
- How do you define a "session" — one thread per session, grouped by day, or patient-initiated?
- Can a single messaging session involve multiple patients (e.g. a parent asking about two children)?
- Do you need to capture diagnosis codes, service type, or other clinical details per encounter?

**What the answers drive:**

| Situation | Approach |
| --- | --- |
| No billing intent | No Encounter resource needed; thread stands alone |
| Billing needed, one patient per session | Create an Encounter per session; link to the thread header via Communication.encounter |
| One thread = one session | Straightforward 1:1 mapping of thread to Encounter |
| Rolling interaction model (e.g. continuous text thread, grouped by day) | One Encounter spans multiple threads or a time window; multiple thread headers link to the same Encounter |
| Multiple patients can be in one session | Create a parent session Encounter + one child Encounter per patient via Encounter.partOf; clinical details live on child Encounters |
| Clinical documentation needed for billing | Populate Encounter.participant, Encounter.reasonCode, and Encounter.serviceType per patient Encounter |

## Related Integrations

The external messaging channels Medplum integrates with first-party — both deliver through FHIR Communication resources, so they slot into the thread model above:

- **Twilio SMS** — send and receive SMS via the $send-sms-twilio operation.
- **eFax** — send and receive faxes via the $send-efax and $receive-efax operations.